

Queues (Assignment Solutions)

Question 1 :

```
int timeRequiredToBuy(vector<int>& tickets, int k) {  
    // {tickets, og_index}  
    queue<pair<int, int>> q;  
    int n = tickets.size();  
    for(int i = 0; i < n; i++) {  
        q.push({tickets[i], i});  
    }  
    int time = 0;  
    while(!q.empty()) {  
        auto [num_tickets, og_i] = q.front();  
        q.pop();  
        num_tickets--;  
        time++;  
        if(og_i == k && num_tickets == 0) {  
            return time;  
        }  
        if(num_tickets > 0) {  
            q.push({num_tickets, og_i});  
        }  
    }  
    return time;  
}
```

Question 2 :

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {  
    int totalgas = 0;  
    int currgas=0;  
    int startIndex =0;  
  
    for(int i=0;i<gas.size();i++)  
    {
```

```
        // (gas-cost) add in total and currgas.

        currgas +=gas[i]-cost[i];
        // if ans exists value net cost/gas should be positive.
        totalgas+=gas[i]-cost[i];

        //if currcost is -ve then reintialize startindex.
        if(currgas<0){
            startIndex = i+1;
            currgas=0;
        }
    }
    return totalgas>=0 ? startIndex:-1;
}
```

Question 3 :

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {

    int totalgas = 0;
    int currgas=0;
    int startIndex =0;

    for(int i=0;i<gas.size();i++)
    {
        // (gas-cost) add in total and currgas.

        currgas +=gas[i]-cost[i];
        // if ans exists value net cost/gas should be positive.
        totalgas+=gas[i]-cost[i];

        //if currcost is -ve then reintialize startindex.
        if(currgas<0){
            startIndex = i+1;
            currgas=0;
        }
    }

    return totalgas>=0 ? startIndex:-1;
}
```

```
}
```

Question 4 :

```
class LRUCache {
private:
    struct Node {
        int key, value;
        Node* next;
        Node* prev;
        Node(int k, int v) : key(k), value(v), next(nullptr), prev(nullptr) {}
    };

    unordered_map<int, Node*> map;
    int capacity;
    Node* head;
    Node* tail;

    void pushToHead(Node* node) {
        remove(node);
        add(node);
    }

    void add(Node* node) {
        node->next = head->next;
        node->next->prev = node;
        node->prev = head;
        head->next = node;
    }

    void remove(Node* node) {
        Node* prev = node->prev;
        Node* next = node->next;
        prev->next = next;
        next->prev = prev;
    }
}
```

```
public:
    LRUCache(int capacity) : capacity(capacity) {
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head->next = tail;
        tail->prev = head;
    }

    ~LRUCache() {
        Node* current = head;
        while (current != nullptr) {
            Node* next = current->next;
            delete current;
            current = next;
        }
    }

    int get(int key) {
        if (map.find(key) == map.end()) {
            return -1;
        }
        Node* node = map[key];
        pushToHead(node);
        return node->value;
    }

    void put(int key, int value) {
        if (map.find(key) != map.end()) {
            Node* node = map[key];
            node->value = value;
            pushToHead(node);
        } else {
            if (map.size() >= capacity) {
                Node* lru = tail->prev;
                map.erase(lru->key);
                remove(lru);
            }
        }
    }
};
```

```
        delete lru;  
    }  
    Node* node = new Node(key, value);  
    map[key] = node;  
    add(node);  
}  
}  
};
```